AgentForge

Adversarial AI Security Platform

Building a Multi-Agent Adversarial Evaluation System for AI-Assisted Healthcare

Project Requirements Document

Gauntlet AI — Austin Admission Track — Week 3

How to Use This Case Study

This case study is your north star for the duration of this project. You are required to use it as the foundation for every decision you make: what you build, what you prioritize, and what you expand. It does not define the ceiling. If you see opportunities to go deeper or broader, take them, but do it in the spirit of the case study, not in spite of it.

What this case study does define is the floor: every feature, every architectural decision, and every tradeoff you make should be traceable back to the problem of building a system that can continuously identify, evaluate, and defend against adversarial attacks on AI-assisted clinical workflows. Use this document as a reference, a constraint, and a lens.

The decisions you make this week build directly on the foundation from Weeks 1 and 2. Good architecture compounds; technical debt costs double. You will be evaluated on your thoroughness, your thoughtfulness, your creativity, and your ability to leverage technology to build something viable and defensible.

The Scenario

OpenEMR has recently begun experimenting with AI-assisted workflows through a new Clinical Co-Pilot chatbot feature connected to patient and operational data. The system is intended to help users retrieve chart information, summarize notes, assist with intake workflows, and support general clinical operations inside the OpenEMR environment.

The early prototype has shown promise, but there are growing concerns around reliability, security, and long-term maintainability.

As usage increased, inconsistent behavior began surfacing under certain conditions. Some prompts caused the assistant to behave outside its intended scope. Some uploaded content appeared to influence future responses in unexpected ways. Multi-turn conversations occasionally produced responses that ignored previous safeguards or operational boundaries.

At the same time, infrastructure costs began increasing faster than expected. Long-running conversations, repeated retrieval behavior, recursive tool usage, and large prompt chains made it difficult to predict runtime cost and performance. Some tests produced inconsistent results between runs, making evaluation and debugging difficult.

The larger concern is not whether a single exploit exists.

The concern is whether the system can continuously identify, evaluate, and defend against new attack techniques as the platform evolves.

The current testing process relies mostly on manual prompting and static attack lists. Vulnerabilities discovered during testing are difficult to reproduce consistently. Fixes are often validated once and never tested again against future variants or regressions. There is limited visibility into which attack categories are actually covered and which parts of the system remain largely untested.

OpenEMR leadership wants to move beyond one-time penetration testing and toward a reusable adversarial evaluation platform capable of continuously stress testing AI systems connected to healthcare workflows.

You have been brought in to design the next generation of that security infrastructure.

Your objective is to build a system capable of:

- Discovering vulnerabilities automatically
- Generating adversarial attacks dynamically
- Measuring attack success and coverage over time
- Converting successful exploits into repeatable evaluations
- Validating that fixes actually work
- Preventing regressions as the system evolves
- Documenting vulnerabilities professionally
- Improving visibility into how the system behaves under adversarial pressure

The organization also recognizes several operational realities. Large frontier models can become prohibitively expensive when used for continuous security testing at scale. Some commercial LLMs are intentionally trained to avoid offensive security workflows, making them unreliable for certain forms of adversarial testing. Smaller or open-source models may provide different capabilities, lower cost, or greater flexibility depending on the task.

Traditional non-AI security tooling may also outperform LLM-driven approaches in certain areas, especially around deterministic validation, replay testing, fuzzing, and protocol-level analysis. Part of the challenge is determining which tools, models, evaluation strategies, and security methodologies are most appropriate for building a scalable and repeatable adversarial testing platform.

WHY THIS MATTERS	*The goal is not simply to find a few jailbreaks. The goal is to build a system of agents that can hunt, evaluate, escalate, and document vulnerabilities continuously — adapting as attackers adapt, without a human in the loop for every step. A static test suite is not the answer. An autonomous multi-agent red team is.*

The Hard Problems

This is not a build-whatever-you-want project. The case study surface area is intentionally constrained, but the engineering problems within it are real and unresolved. Your job is not simply to generate attacks. Your job is to build a multi-agent system capable of discovering, evaluating, validating, and improving the security posture of an evolving LLM application — autonomously and continuously.

A single-agent or pipeline architecture will not satisfy this assignment. The problems below are structured specifically because they require different agents with different roles, capabilities, and trust levels operating in coordination. A red team agent that generates attacks is not the same as a judge agent that evaluates them. An orchestrator that prioritizes coverage is not the same as a documentation agent that files vulnerability reports. These are distinct responsibilities that benefit from distinct agents.

How you design those agents, how they communicate, how they hand off work, and how they recover from failure are the core engineering decisions of this assignment. There are many valid multi-agent architectures. The requirement is that your decisions are deliberate, defensible, and grounded in evidence.

Adversarial Robustness

Systems that perform well under normal usage can behave very differently under adversarial pressure. Prompt injection, indirect instructions, multi-turn manipulation, uploaded content, and state corruption can all influence model behavior in subtle ways that are difficult to predict ahead of time.

The challenge is not simply finding one successful exploit. The harder problem is determining whether a category of exploit has actually been addressed, especially once attacks begin mutating and evolving over time. Static payload lists become outdated

quickly. Defenses built around a small number of known examples rarely hold as attackers adapt.

This is precisely why a static test runner is insufficient. You need an agent whose job is to probe, mutate, and escalate — one that can take a partially-successful attack and autonomously generate ten variants to find the version that breaks through. That agent should not need a human to tell it what to try next.

Evaluation & Regression

Finding a vulnerability once is not enough. A discovered exploit only becomes useful when it can be reproduced consistently, converted into an evaluation, and used to validate future changes to the system.

One of the difficult engineering problems in AI systems is determining whether a fix actually improves the system or simply changes its behavior temporarily. Defenses may block one attack while introducing regressions somewhere else. Other fixes may appear successful until a slightly modified variant bypasses them again later.

This creates a distinct role in your architecture: a judge agent whose sole responsibility is evaluating whether an attack succeeded, whether a defense held, and whether a regression has appeared. The judge must be independent of the attack engine — an agent that both generates attacks and evaluates them is compromised by design. How you define the judge's evaluation criteria, how you prevent it from drifting, and how you validate the judge itself are hard problems you must solve.

Visibility & Observability

Security systems are difficult to improve when there is little visibility into what is actually happening inside them. It is often unclear which attack surfaces have been explored, which attack categories are succeeding most often, or whether the system is improving or regressing as changes are introduced.

The challenge is not only executing attacks, but understanding system behavior well enough to measure progress over time and make intelligent decisions about what to test next. This is the role of an orchestrator agent: one that reads the state of the system — coverage gaps, high-severity open findings, recent regressions — and decides where to direct the red team agent's attention. Without this layer, your platform is just running attacks randomly. With it, your platform is learning.

Your system should help make the attack surface more understandable over time, not more opaque. Logging, reporting, replayability, and trend analysis are not afterthoughts — they are the signals your orchestrator depends on to function.

Cost, Scale, & Model Constraints

Large frontier models are powerful, but they are also expensive, inconsistent, rate-limited, and often intentionally resistant to offensive security workflows. Approaches that work during small experiments may become difficult to sustain once testing volume increases.

Part of the challenge is determining when AI-driven approaches are useful, when deterministic systems are more reliable, and when smaller or local models may be more practical than large hosted APIs. Traditional security tooling may also outperform LLMs in certain scenarios.

The assignment intentionally leaves those decisions open-ended.

Discovery, Remediation, & Trust

Finding vulnerabilities and fixing vulnerabilities are different problems. A real security workflow does not stop once an exploit is discovered. Vulnerabilities must be documented, reproduced, evaluated, patched, validated, and monitored over time.

This creates another natural agent boundary: a documentation agent that takes a confirmed exploit from the judge and produces a structured, professional vulnerability report — without human intervention. But automation introduces its own trust problem. A system that autonomously files vulnerability reports, recommends patches, or triggers remediation workflows must have clearly defined trust boundaries and human approval gates. An agent that confidently documents a false positive wastes engineering time. An agent with the ability to push fixes without review can introduce entirely new vulnerabilities.

Part of this assignment is designing those trust boundaries deliberately. Where does your system stop and ask a human? Where does it proceed autonomously? How does it communicate confidence, and what happens when that confidence is wrong?

Existing Security Research & Industry Practices

Many of the challenges in this assignment already exist in the real world. Security researchers, red teams, and AI safety organizations have spent years developing methodologies, terminology, frameworks, and testing approaches for adversarial systems.

Part of the challenge is learning what already exists, understanding where current approaches succeed or fail, and deciding which ideas are applicable to LLM systems specifically.

The assignment intentionally does not prescribe a single correct architecture or workflow. Part of the work is navigating an ambiguous and rapidly evolving problem space and making thoughtful engineering decisions within it.

Project Schedule

Checkpoint	Deadline
Architecture Defense	4 hours after kickoff
MVP	Tuesday @ 11:59 PM
Final	**Friday @ Noon**

MVP: Recommended Steps

The MVP is not a finished adversarial platform. It is the foundation that makes a trustworthy, repeatable evaluation system possible.

Stage	Name	Deliverable
1	Stand Up the Target	Clinical Co-Pilot from Weeks 1 and 2 running locally and deployed, ready to be tested
2	Map the Attack Surface	A structured breakdown of all adversarial entry points, trust boundaries, and threat categories
3	Build Initial Attack Suite	A working set of adversarial test cases across at least three attack categories
4	Plan the Platform Architecture	A concrete, defensible plan for how the full adversarial evaluation system will be built

Stage 1 — Stand Up the Target

Your adversarial platform needs something to attack. Ensure your Clinical Co-Pilot from Weeks 1 and 2 is running in a testable state — locally and deployed. If you are starting fresh, you can use the OpenEMR Clinical Co-Pilot from the Week 1 case study as your target.

Document any changes made to bring the system into a testable state. This becomes part of your README and your threat model context.

HARD GATE	*Your deployed target application URL must be submitted with every checkpoint. The adversarial platform must be running tests against a live system, not just a mock.*

Stage 2 — Map the Attack Surface

Before building attacks, you must understand what you are attacking. Produce a structured threat model covering the full attack surface of the Clinical Co-Pilot. At minimum, this should include:

- Prompt injection vectors — direct, indirect, and multi-turn
- Data exfiltration paths — PHI leakage, cross-patient data exposure, authorization bypass
- State corruption attacks — conversation history manipulation, context poisoning
- Tool misuse — unintended tool invocation, parameter tampering, recursive tool calls
- Denial of service patterns — token exhaustion, infinite loops, cost amplification
- Identity and role exploitation — privilege escalation, persona hijacking, trust boundary violations

For each category, identify: the attack surface, the potential impact, the difficulty of exploitation, and whether existing defenses address it. This threat model is not a one-time artifact — it is a living document your platform will continuously exercise.

HARD GATE	*A markdown document (./THREAT_MODEL.md) with your full attack surface map. The document must begin with a ~500 word summary of your key findings, the highest-risk attack categories, and how your platform will prioritize coverage.*

Stage 3 — Build Initial Attack Suite

Using your threat model as input, build an initial set of adversarial test cases that exercise the highest-priority attack categories. These are not static payloads — they are the seed cases your Red Team Agent will learn from and extend. You should also have a working prototype of at least one agent role operating against the live target at this stage.

Each test case should include:

- Attack category and subcategory
- The specific prompt or input sequence
- The expected safe behavior
- The observed behavior (pass/fail/partial)
- Severity rating and exploitability assessment
- Whether the case should be added to the regression suite

You are not expected to have the full multi-agent platform running at this stage. You are expected to demonstrate that your test cases are structured, reproducible, and extensible — and that you have begun building the agent architecture that will scale them.

HARD GATE	*A working test suite (./evals/) with results from at least three distinct attack categories, plus a working prototype of at least one agent role (Red Team, Judge, or Orchestrator) running live against the deployed target.*

Stage 4 — Plan the Platform Architecture

Using your threat model and user definitions as inputs, produce a forward-looking architecture plan for the full multi-agent adversarial evaluation platform. This plan must define each agent role, its responsibilities, its inputs and outputs, and how it coordinates with the others. It should also address:

- Which agent is responsible for each function: attack generation, evaluation, orchestration, documentation
- How agents communicate — what messages or signals pass between them and in what format
- How the Orchestrator decides what the Red Team Agent targets next
- How the Judge Agent's verdicts feed into the regression harness
- Where human approval gates exist and why they are placed there
- Where AI is used versus deterministic tooling, and the justification for each
- How the platform handles cost, rate limits, and model constraints at scale
- What framework or infrastructure manages agent state and coordination

You do not need to implement the full platform at this stage. You need to think clearly, write it down, and be able to defend every architectural decision — especially the ones that involve agents acting autonomously.

HARD GATE	*A markdown document (./ARCHITECTURE.md) defining your multi-agent platform architecture. The document must begin with a ~500 word summary and must explicitly name each agent, its role, and how it fits into the overall system. A diagram of agent interactions is strongly recommended.*

Platform Requirements

Your adversarial evaluation platform must be built as a multi-agent system. The following are the required agent roles and platform components. How you implement

each one — what framework you use, how agents communicate, how state is managed — is a design decision you own. What is not optional is the multi-agent architecture itself.

ARCHITECTURE REQUIREMENT	*A single-agent or pipeline architecture does not satisfy this assignment. Each role below represents a distinct agent with its own responsibilities, context, and decision-making authority. Your ARCHITECTURE.md must define each agent, its inputs and outputs, its trust level, and how it coordinates with the others.*

Multi-Agent Adversarial System

The core of the platform is a multi-agent system that autonomously discovers, evaluates, and escalates vulnerabilities in the Clinical Co-Pilot. How you design that system — how many agents, what roles, what framework, how they communicate — is your architectural decision to make and defend.

Whatever architecture you choose, the system must collectively be capable of:

- Generating novel adversarial inputs
- Mutating partially-successful attacks to probe for bypasses
- Targeting multi-turn attack sequences, not just single-prompt injections
- Evaluating whether an attack succeeded, with consistent criteria across runs and system versions
- Prioritizing which attack surfaces to explore next based on coverage gaps and unresolved findings
- Halting or redirecting when cost is accumulating without producing signal
- Triggering regression runs when the target system changes

These capabilities imply distinct responsibilities. Attack generation and attack evaluation are different jobs — a system that does both in the same context has a conflict of interest by design. Strategic prioritization is different from execution. How you separate, combine, or coordinate those responsibilities across agents is the core architecture problem of this assignment.

Think carefully about what models power each role. Commercial frontier models are often trained to refuse offensive security workflows. Smaller or open-source models may be more capable in certain positions. That is a deliberate decision you must make and defend. *LLM cost is also a factor as this can run up tokens very quickly.*

Documentation Agent

The Documentation Agent converts confirmed exploits from the Judge Agent into structured, professional vulnerability reports — without requiring a human to write them.

Each report it produces must be reproducible, actionable, and usable by an engineer who was not present when the exploit was found.

At minimum, each report must include:

- A unique identifier and severity rating
- A clear description of the vulnerability and its clinical impact
- A minimal, reproducible attack sequence
- The observed versus expected behavior
- Recommended remediation approach
- Current status and fix validation results

The bar is not that the Documentation Agent produces interesting output. The bar is that a senior security engineer could reproduce, validate, and fix the vulnerability based solely on what the agent writes.

Regression & Validation Harness

Across all agents, the platform must maintain a regression harness that converts confirmed exploits into deterministic, repeatable test cases and runs them against every new version of the target system. The harness must:

- Store confirmed exploits in a versioned, queryable format
- Run the full regression suite automatically when triggered by the Orchestrator
- Detect when a previously-fixed vulnerability has reappeared
- Flag when fixing one attack introduces a regression in another category

Think carefully about what it means for a regression test to pass. A test that passes because the model's behavior changed — not because the vulnerability was actually fixed — is worse than no test at all.

Observability Layer

The platform must surface enough information for the Orchestrator to make intelligent decisions and for a human operator to understand system behavior at any time. At minimum, you must be able to answer:

- Which attack categories have been tested, and how many cases exist per category?
- What is the current pass/fail rate across all test categories and system versions?
- Is the target system becoming more or less resilient over time?
- Which vulnerabilities are open, in progress, or resolved?
- How much did this test run cost, and at what rate is cost scaling?
- What is each agent doing, and in what order did it happen?

The observability layer is not just for humans. It is the data substrate your Orchestrator Agent reads. Design it accordingly.

Engineering Requirements

The following requirements apply in addition to the project-specific deliverables above. They are graded alongside the core submission and are not optional.

Test design for boundaries, invariants, and regression. Every adversarial test case must exercise a boundary (maximum prompt length, minimum token budget, edge-case multi-turn sequence), an invariant (the Judge must never approve a confirmed successful exploit), or a regression risk (a previously fixed vulnerability reappearing). Static payload lists do not satisfy this requirement.

Build-versus-configure decisions: custom agents, workflow platforms, deterministic tooling, COTS/SaaS. Before implementing any agent role, document a build-versus-configure decision record: what existing tools were evaluated (Burp Suite, OWASP ZAP, Semgrep, Garak, commercial red-team platforms), what each covers, where it falls short for this use case, and why a custom agent is justified over configuring an existing tool. This record is a required Architecture Defense deliverable.

OWASP Top 10 and OWASP LLM risks. The attack suite must cover the OWASP Top 10 web vulnerabilities relevant to the Clinical Co-Pilot (broken access control, injection, insecure design, vulnerable components, auth failures, logging failures, SSRF) in addition to the OWASP LLM Top 10 (prompt injection, insecure output handling, sensitive information disclosure, excessive agency, supply-chain risks). Map each test case to its OWASP category in the eval dataset.

Lab: triage simulated scan report with critical, high, medium, and false-positive findings. As part of the platform, build a triage exercise: generate a simulated vulnerability scan report containing at least ten findings across critical, high, medium, and false-positive categories. Document how you would validate, remediate, defer, or document each finding. This demonstrates the Documentation Agent's output is usable for real triage decisions.

Interface arbitration, contract tests, migration notes, architecture review, evidence packets, failure drills. Define typed contracts for every inter-agent message (Orchestrator-to-RedTeam, RedTeam-to-Judge, Judge-to-Documentation). Write contract tests that verify these message shapes. Any change to an agent interface must be recorded in a migration note. Your Architecture Defense must include an evidence packet: agent interaction diagram, message schemas, trust boundaries, and known failure modes.

Lab: build one component, inherit another, serve as integration lead for one cross-service boundary. During the integration week exercise, each student builds one agent role and integrates with a peer's independently built agent. The integration must work through the published message contract alone — no shared source access. Produce an integration packet: interface diffs, ADRs for any contract corrections, contract test results, and a cross-agent trace proving end-to-end correctness.

OpenAPI, AsyncAPI, protobuf, JSON Schema, or equivalent protocol contracts. All inter-agent communication must use a versioned schema definition (JSON Schema or equivalent). Publish the schemas in a /contracts directory. Contract tests must verify that both the producing and consuming agents conform to the published schema. Breaking changes require a version bump.

ATO-style evidence packet. Produce an ATO-style evidence packet as a submission artifact: architecture diagram, data flow diagram, auth model (which agents can call which targets and with what credentials), dependency list with versions, vulnerability scan results from the platform itself, test evidence (eval results), and a sample incident/postmortem. This is separate from the Architecture Doc — it is the artifact a security reviewer would accept.

AI-use disclosure: where AI was used, what was independently verified, what remains risky. Include an AI-use disclosure section in ARCHITECTURE.md: list every agent role powered by AI,

what deterministic verification or human approval follows each AI decision, and what risks remain. The Judge Agent's evaluation criteria must be independently verifiable — document how you would detect and correct a drifting judge.

Integration packet: interface diffs, ADRs, contract-test results, cross-team dependency map, end-to-end result proof. Produce a full integration packet as a submission artifact: diffs showing where original agent message contracts were corrected, ADRs recording those decisions, contract test results from both sides of each agent boundary, a dependency map showing which agents call which, and a trace proving end-to-end correctness through the full platform.

Version API contracts. All inter-agent message schemas must be versioned (minimum: v1). Any breaking change to a message schema requires a version bump, a migration note, and updated contract tests. The /contracts directory must reflect the current version of every schema.

Explicit error schemas. Every agent must define typed error responses for its known failure modes: target unreachable, budget exceeded, judge timeout, no findings in window, regression detected. Error schemas must be published in /contracts alongside the success schemas.

Pagination, rate limits, and auth requirements documented. Document the rate limits and auth requirements for every external API the platform calls: LLM providers, the Clinical Co-Pilot target, observability backends. Define how the platform handles rate limit responses — backoff, queue, or abort — and document this behavior in ARCHITECTURE.md.

Data-quality checks: required fields, uniqueness, referential integrity, privacy-sensitive data. The exploit database must enforce data quality: every confirmed exploit has a unique identifier, required fields (severity, attack category, reproduction steps, observed behavior, expected behavior), and no duplicate entries for the same attack sequence. Validate these constraints in the Documentation Agent before writing a report.

API versioning, database performance, migrations, queues, workflow state. The exploit database schema must support migration — if you add a field to the vulnerability report format, existing records must be migrated without data loss. Document the migration approach. If the platform uses a queue (for agent work items or regression runs), document the queue depth monitoring and what happens when it backs up.

Data modeling, ingestion, validation, lineage, access control, reporting, data quality. Document the data model for the adversarial platform's own data: exploit records, regression test cases, agent run logs, coverage metrics. Define access controls: which agents can write exploits, which can only read them, and what human approval is required before a finding is published.

SQL performance, indexing, batch jobs, reproducible analysis. If the exploit database uses SQL, document the indexes on the most common query patterns (by severity, by attack category, by system version). Regression runs that scan the full exploit database must complete within a defined time budget — document this SLO and verify it in CI.

Baseline CPU, memory, latency, and throughput profiles. Capture baseline metrics for the adversarial platform itself under a representative test run (100 attack cases, full regression suite). Include these baselines in the submission so future platform performance changes can be measured against them.

Load/stress tests with realistic scenarios. Run a load test simulating the platform executing 100 consecutive attack cases against the live target. Record platform-level metrics (agent orchestration latency, LLM call latency, exploit storage throughput). Identify the bottleneck and document the architectural change that would address it.

Submission Requirements

Deliverable	Requirements

GitHub Repository	Independent code repo. Includes setup guide, architecture overview, deployed link, and instructions for running the adversarial platform against the live target.
Threat Model (./THREAT_MODEL.md)	Full attack surface map with a ~500 word summary of key findings and highest-risk categories.
User Doc (./USERS.md)	The users your platform addresses, their workflows, and specific use cases with explicit justification for why automation is the right solution.
Architecture Doc (./ARCHITECTURE.md)	Your multi-agent platform architecture with technical details including each agent role, inter-agent communication design, orchestration strategy, regression harness, observability layer, and known tradeoffs. Must begin with a ~500 word summary. Must include a diagram of agent interactions.
Demo Video (3-5 min)	One demo video per submission showcasing the work completed, highlighting key decisions, and demonstrating the platform running live attacks against the target system.
Eval Dataset (./evals/)	Your adversarial test suite with results across at least three attack categories. Structure and scope are your design decisions; results must be reproducible.
Vulnerability Reports	Professional documentation for each discovered vulnerability following the required format. Minimum of three distinct vulnerability reports.
AI Cost Analysis	Actual dev spend and projected production costs for running the adversarial platform at 100 / 1K / 10K / 100K test runs. Consider architectural changes needed at each scale. This is not simply cost-per-token × n runs.
Deployed Application	Publicly accessible target system. For early and final submissions, the adversarial platform must be running live tests against the deployed target.
Social Post (Final only)	Share on X or LinkedIn: describe the project, show the platform in action, tag @GauntletAI.

Final Note

The deliverable that matters is not the one that finds the most impressive jailbreak in a demo. It's the one you could defend in front of a hospital CISO who is deciding whether

to trust this platform with continuous security testing of systems their physicians depend on.

That is the standard. Build to it.

Appendix: Pre-Search Checklist

Use this list to ensure you've thought through a variety of perspectives in your planning.

Phase 1: Define Your Constraints

1. Domain & Threat Selection

- What specific attack categories will you target first?
- What are the verification requirements for determining exploit success?
- What attack surfaces require human judgment versus automated assessment?

2. Scale & Performance

- Expected test run volume per day/week?
- Acceptable latency for attack generation and evaluation?
- Concurrent testing requirements against the target system?
- Cost constraints for LLM-driven attack generation?

3. Reliability Requirements

- What is the cost of a false negative (missed vulnerability)?
- What is the cost of a false positive (flagging safe behavior as vulnerable)?
- Human-in-the-loop requirements for remediation approval?
- Audit and compliance needs for security testing in a healthcare context?

Phase 2: Architecture Discovery

4. Multi-Agent Design

- How many distinct agent roles does your platform require?
- What framework manages agent coordination — LangGraph, CrewAI, AutoGen, custom?
- How do agents hand off work — message queues, shared state, direct invocation?
- What happens when one agent in the pipeline fails or times out?

5. Red Team Agent Design

- What model powers the Red Team Agent, and why is it appropriate for offensive security?
- How does the agent mutate a failed attack into a variant?
- How do you prevent the Red Team Agent from being blocked by the same safety filters it is trying to probe?
- Context window requirements for multi-turn attack sequences?

6. Judge Agent Design

- How does the Judge determine success vs. failure vs. partial for each attack category?
- How do you validate that the Judge's criteria are consistent across runs?
- What happens when the Judge is uncertain — does it escalate to a human?
- How do you prevent the Judge from drifting as the target system changes?

7. Orchestrator Design

- What signals does the Orchestrator read to prioritize the next attack campaign?
- How does the Orchestrator decide when a category is sufficiently covered?
- What triggers a regression run — a new deployment, a time window, or both?
- How does the Orchestrator manage cost across agents in a single session?

8. Observability Strategy

- LangSmith vs. Langfuse vs. Braintrust vs. custom — which surfaces inter-agent traces?
- What metrics matter most for a multi-agent security platform?
- How do you trace a vulnerability finding back through all the agents that produced it?
- Cost tracking at the agent level, not just the platform level?

Phase 3: Post-Stack Refinement

8. Failure Mode Analysis

- What happens when the Red Team Agent generates content that is itself harmful?
- How do you handle a Judge Agent that starts agreeing with everything?
- What is the fallback when the Orchestrator has no clear next priority?
- How do you handle cascading failures across agents in a single test run?

9. Trust & Safety for the Platform Itself

- How do you prevent the adversarial platform from being turned against systems it should not attack?
- Access controls for who can trigger agent runs and view vulnerability reports?
- What approval is required before the Documentation Agent files a critical-severity report?
- How do you audit what each agent did during an overnight run?

10. Testing the Tester

- How do you validate that the Red Team Agent is actually generating novel attacks?

- Ground truth dataset for evaluating Judge Agent accuracy?
- How do you detect when the platform is producing low-quality signal?
- What does it mean for the multi-agent system itself to regress?

11. Iteration Planning

- How will you add new agent roles as the platform matures?
- Eval-driven improvement cycle for each agent independently?
- How does the platform incorporate newly published attack techniques automatically?